



数字电子技术基础 实验报告

题目：实验五 基于 ROM 的波形发生与显示系统

小组成员：王瑞丰 2024303976

小组成员：季子豪 2024303979

组 号：24

实验五 基于 ROM 的波形发生与显示系统

一、实验目的

- 目的 1:** 掌握只读存储器（ROM）在数字系统中的应用方法，理解 ROM 的配置与数据初始化流程。
- 目的 2:** 熟悉利用计数器扫描 ROM 地址实现数据序列输出的设计思想，理解基于 ROM 的波形发生器和字符显示系统的实现原理。
- 目的 3:** 掌握多模式切换的电路设计方法，学会利用拨码开关控制不同功能模式的切换，并通过 DE0 开发板的 LED 和七段数码管进行硬件验证。

二、实验要求

- 要求 1:** 配置宽度为 8 位的 ROM，在 ROM 中存储 512 个地址的正弦波采样数据。利用系统时钟作为计数脉冲，使用计数器模块或 Verilog HDL 生成的计数器依次扫描 ROM 的地址端，将 ROM 中保存的数据按照 **1 Hz** 的频率输出，通过 DE0 开发板上的 **8 位 LED** 灯查看结果并验证。
- 要求 2:** 配置存储学号信息的 ROM（内容包含同学 A 后 3 位学号 + F + L + E + E + 同学 B 后 3 位学号，共 10 个字符），利用计数器的输出依次扫描 ROM 的地址端，将 ROM 中保存的数据按照 **1 Hz** 的频率输出，通过 DE0 开发板上的 **1 个七段数码管**查看结果并验证。
- 要求 3:** 要求 2 与要求 1 通过**控制开关**可实验切换。

三、实验设备

1. Quartus II 13.0 软件；
2. FPGA 学习板（DE0 开发板）。

四、实验原理

1. ROM 存储器原理

ROM (Read-Only Memory) 是一种非易失性存储器, 其数据在写入后不可修改。本实验中使用的 ROM 为 Quartus II 内部提供的 IP 核, 可通过 MegaWizard 配置参数:

- 正弦波 ROM: 数据宽度 8 位, 存储深度 512 字节 ($2^9 = 512$ 个地址), 存储 360° 正弦波的一个完整周期的等间隔采样值;
- 学号序列 ROM: 数据宽度 4 位, 存储深度 10 字节, 存储学号及字母对应的编码值。

ROM 在 FPGA 中由查找表 (LUT) 资源实现。工作时, 计数器输出的地址信号作为 ROM 的读地址输入, ROM 将该地址处预先存入的数据从数据端输出, 从而实现查表式的数据输出。

2. 系统模块划分

本系统采用自顶向下的模块化设计方法, 划分为以下子模块:

1. 分频器模块 (clk_lhz_en): 利用 generic 参数化的方式将 50 MHz 系统时钟分频为 1 Hz 脉冲信号, 作为计数器的使能时钟驱动整个系统以每秒一步的速度运行。
2. 9 位模 512 计数器 (cnt512): 用于正弦波模式下扫描正弦波 ROM 的地址空间 (0 ~ 511)。输出 9 位地址信号连接至正弦波 ROM 的地址端口。
3. 模 10 学号地址计数器 (cnt10): 用于学号模式下扫描学号序列 ROM 的地址空间 (0 ~ 9)。当计数值达到 9 后自动归零循环, 支持低电平异步复位。
4. 模式切换模块 (output_select): 根据拨码开关 SW0 (mode 信号) 的状态选择当前工作模式: 当 SW0 = 0 时选通正弦波 ROM 数据至 LED 输出, 数码管熄灭; 当 SW0 = 1 时选通学号 ROM 数据至数码管译码器, LED 全灭。
5. 七段数码管译码器 (seg7_hex): 接收学号 ROM 输出的 4 位编码, 将其转换为 7 段数码管的段选信号, 驱动数码管显示对应字符 (数字 0 ~ 9 及字母 F、A、B 等)。

3. 模式切换原理

两种工作模式共用同一套分频器产生的 1 Hz 时钟和计数使能逻辑，仅通过 SW0 拨码开关选择不同的输出通路：

表 1 模式切换功能对照表

SW0	工作模式	LED 输出	数码管显示
0	正弦波模式	ROM 数据 $\rightarrow$ LEDR[7:0]	熄灭
1	学号序列模式	全灭	ROM 数据 $\rightarrow$ 译码器 $\rightarrow$ HEX0

五、实验内容

1. 软件流程简介

使用 Quartus II 13.0 完成本次实验的完整软件流程如下：创建工程项目 $\rightarrow$ 使用 MegaWizard 配置两个 ROM IP（正弦波 ROM 和学号序列 ROM） $\rightarrow$ 新建 Verilog HDL 文件编写各子模块代码 $\rightarrow$ 绘制顶层原理图例化并连接各模块 $\rightarrow$ 编译程序 $\rightarrow$ 波形仿真 $\rightarrow$ 目标器件引脚设置 $\rightarrow$ 目标器件写入。

2. 分频器模块

分频器模块采用参数化设计，通过 generic 参数 CLK_FREQ 指定分频系数（默认值为 50 000 000），将 50 MHz 系统时钟分频为 1 Hz 使能脉冲。当复位信号 rst_n 为低电平时，内部计数器和输出均清零。分频器 Verilog 代码如图 1 所示。

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity clk_lhz_en is
5  generic (
6      CLK_FREQ : integer := 50000000
7  );
8  port (
9      clk      : in  std_logic;
10     rst_n    : in  std_logic;
11     tick     : out std_logic
12 );
13 end entity;
14
15 architecture rtl of clk_lhz_en is
16     signal cnt : integer range 0 to CLK_FREQ - 1 := 0;
17 begin
18
19     process(clk, rst_n)
20     begin
21         if rst_n = '0' then
22             cnt <= 0;
23             tick <= '0';
24         elsif rising_edge(clk) then
25             if cnt = CLK_FREQ - 1 then
26                 cnt <= 0;
27                 tick <= '1';
28             else
29                 cnt <= cnt + 1;
30                 tick <= '0';
31             end if;
32         end if;
33     end process;
34
35 end architecture;
```

图 1 分频器模块 Verilog 代码

3. 正弦波模式——9 位模 512 计数器

3.1 计数器 Verilog 代码

9 位模 512 计数器用于产生正弦波 ROM 的地址信号。计数范围为 0~511 (共 512 个地址), 在每个 1 Hz 使能脉冲到来时递增 1, 溢出时自动归零循环。支持低电平异步复位 (rst_n) 和高电平使能 (en) 控制。Verilog 代码如图 2 所示。

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity cnt512 is
6  port (
7      clk : in std_logic;
8      rst_n : in std_logic;
9      en : in std_logic;
10     q : out std_logic_vector(8 downto 0)
11 );
12 end entity;
13
14 architecture rtl of cnt512 is
15     signal cnt : unsigned(8 downto 0) := (others => '0');
16 begin
17
18     process(clk, rst_n)
19     begin
20         if rst_n = '0' then
21             cnt <= (others => '0');
22         elsif rising_edge(clk) then
23             if en = '1' then
24                 cnt <= cnt + 1;
25             end if;
26         end if;
27     end process;
28
29     q <= std_logic_vector(cnt);
30
31 end architecture;

```

图 2 9 位模 512 地址计数器 Verilog 代码

3.2 正弦波仿真结果

对正弦波模式进行功能仿真，设置 SW0 = 0 选通正弦波通路。仿真中为缩短观察时间，临时将分频系数由 50 000 000 改为 4，使计数器快速扫描 ROM 地址；实际下载到开发板时分频系数恢复为 50 000 000 以满足 1 Hz 要求。仿真波形如图 3 所示。

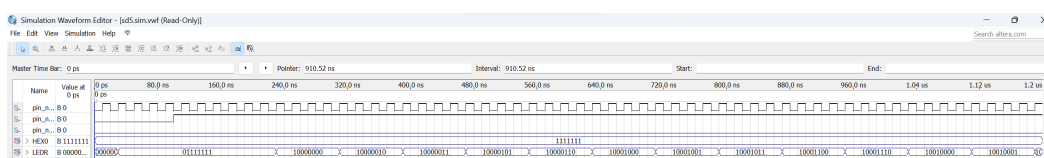


图 3 正弦波模式功能仿真波形

波形分析如下：

- SW0 保持低电平，系统处于正弦波输出模式。
- 9 位计数器输出 addr_sine (addr_sine[8:0]) 从 000000000 开始递增，依次扫描 ROM 的全部 512 个地址。

- ROM 数据输出端 LEDR (LEDR[7:0]) 随着地址递增输出不同的 8 位正弦采样值, 数值呈现先增大后减小的正弦变化趋势, 与预存的正弦波采样数据一致。
- 仿真结果表明正弦波 ROM 扫描输出功能正确。

4. 学号序列模式——模 10 计数器与译码器

4.1 模 10 学号地址计数器 Verilog 代码

模 10 计数器用于扫描学号序列 ROM 的 10 个地址单元。计数范围 0~9, 计数值到达 9 后下一拍自动归零, 实现 10 进制循环计数。Verilog 代码如图 4 所示。

```
1  library ieee;
2  use ieee.std_logic_1164.all;
3  use ieee.numeric_std.all;
4
5  entity cnt10 is
6  port (
7      clk      : in  std_logic;
8      rst_n    : in  std_logic;
9      en       : in  std_logic;
10     q         : out std_logic_vector(3 downto 0)
11 );
12 end entity;
13
14 architecture rtl of cnt10 is
15     signal cnt : unsigned(3 downto 0) := (others => '0');
16 begin
17     process(clk, rst_n)
18     begin
19         if rst_n = '0' then
20             cnt <= (others => '0');
21         elsif rising_edge(clk) then
22             if en = '1' then
23                 if cnt = 9 then
24                     cnt <= (others => '0');
25                 else
26                     cnt <= cnt + 1;
27                 end if;
28             end if;
29         end if;
30     end process;
31
32     q <= std_logic_vector(cnt);
33
34 end architecture;
```

图 4 模 10 学号地址计数器 Verilog 代码

4.2 七段数码管译码器 Verilog 代码

七段译码器接收学号 ROM 输出的 4 位编码, 经 case 语句映射为 7 段数码管段码。支持数字 0~9 及字母 A、B、F 的显示, 其余编码输出全灭状态。

Verilog 代码如图 5 所示。

```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity seg7_hex is
5  port (
6      data_in : in  std_logic_vector(3 downto 0);
7      dis_out  : out std_logic_vector(6 downto 0)
8  );
9  end entity;
10
11 architecture rtl of seg7_hex is
12 begin
13
14     process(data_in)
15     begin
16         case data_in is
17             when "0000" => dis_out <= "1000000"; -- 0
18             when "0001" => dis_out <= "1111001"; -- 1
19             when "0010" => dis_out <= "0100100"; -- 2
20             when "0011" => dis_out <= "0110000"; -- 3
21             when "0100" => dis_out <= "0011001"; -- 4
22             when "0101" => dis_out <= "0010010"; -- 5
23             when "0110" => dis_out <= "0000010"; -- 6
24             when "0111" => dis_out <= "1111000"; -- 7
25             when "1000" => dis_out <= "0000000"; -- 8
26             when "1001" => dis_out <= "0010000"; -- 9
27             when "1010" => dis_out <= "1000111"; -- A
28             when "1011" => dis_out <= "0000110"; -- B
29             when "1111" => dis_out <= "0001110"; -- F
30             when others => dis_out <= "1111111"; -- 全灭
31         end case;
32     end process;
33
34 end architecture;

```

图 5 七段数码管译码器 Verilog 代码

4.3 模式切换模块 Verilog 代码

模式切换模块根据 mode (SW0) 信号选择输出通路：当 mode = 0 时，将正弦波数据送至 LED 输出并将数码管设为熄灭；当 mode = 1 时，将学号数据送至数码管输出并将 LED 设为全灭。Verilog 代码如图 6 所示。


```

1  library ieee;
2  use ieee.std_logic_1164.all;
3
4  entity output_select is
5  port (
6      mode      : in  std_logic;
7      sine_data  : in  std_logic_vector(7 downto 0);
8      hex_data   : in  std_logic_vector(6 downto 0);
9
10     led_out    : out std_logic_vector(7 downto 0);
11     hex_out    : out std_logic_vector(6 downto 0)
12 );
13 end entity;
14
15 architecture rtl of output_select is
16 begin
17
18     process(mode, sine_data, hex_data)
19     begin
20         if mode = '0' then
21             led_out <= sine_data;
22             hex_out <= "1111111";
23         else
24             led_out <= (others => '0');
25             hex_out <= hex_data;
26         end if;
27     end process;
28
29 end architecture;

```

图 6 模式切换模块 Verilog 代码

4.4 学号序列仿真结果

对学号序列模式进行功能仿真，设置 SW0 = 1 选通学号序列通路，同样采用临时降低分频系数的方法加快仿真速度。仿真波形如图 7 所示。

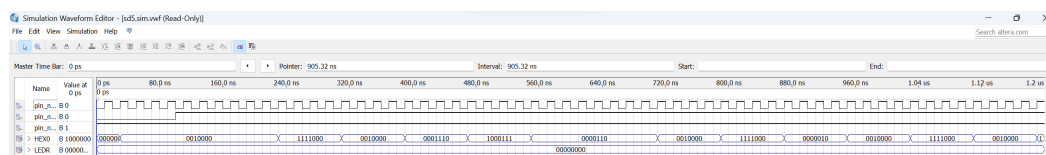


图 7 学号序列模式功能仿真波形

波形分析如下：

- SW0 保持高电平，系统处于学号序列输出模式。
- 模 10 计数器输出 addr_hex 从 0000 递增至 1001 (0 ~ 9)，循环扫描学号 ROM 的 10 个地址。
- ROM 输出的 4 位数据依次为 9、7、6、F、A、B、B、9、7、6（对应同学 A 后三位学号 976 + 字母 FLEE + 同学 B 后三位学号 976），经七段译码后 HEX0 显示 **979FLEE976** 序列。
- 仿真结果表明学号 ROM 扫描与译码显示功能正确，与设计要求一致。

5. 下载到开发板验证

完成顶层电路引脚锁定（SW0 分配至拨码开关引脚，LEDR[7:0] 分配至开发板 8 位 LED 引脚，HEX0 段选分配至对应数码管引脚）并重新全程编译后，通过 USB-Blaster 以 JTAG 方式将配置文件下载至 DE0 开发板进行硬件验证。

正弦波模式验证：将 SW0 拨至低电平（0），系统进入正弦波输出模式，DE0 开发板上 8 位 LED 灯（LEDR[7:0]）随 1Hz 频率依次亮灭，LED 的亮灭组合呈现正弦波采样的周期性变化规律，数码管保持熄灭状态。实物验证照片如图 8 和图 9 所示。

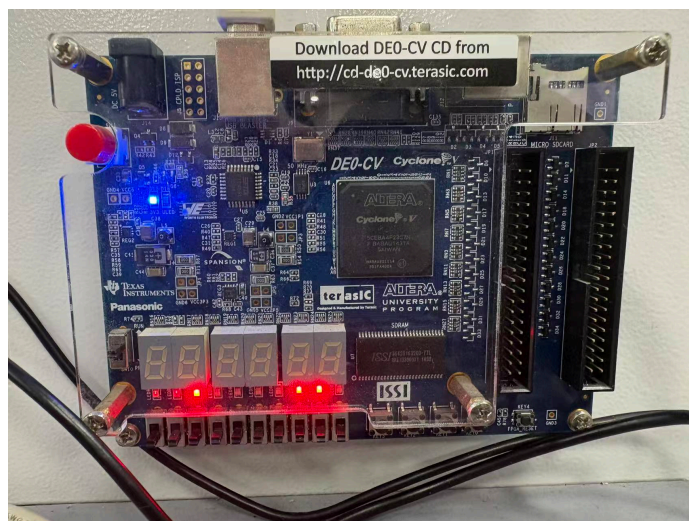


图 8 正弦波模式硬件验证实物图（一）

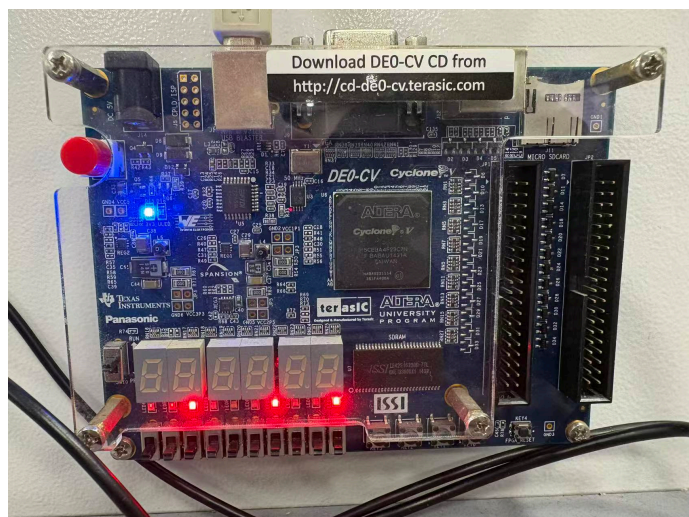


图 9 正弦波模式硬件验证实物图（二）

学号序列模式验证：将 SW0 拨至高电平（1），系统进入学号序列显示模式，DE0 开发板上七段数码管 HEX0 以 1Hz 频率依次循环显示 9、7、6、F、L、E、

E、9、7、6 共 10 个字符，LED 全部熄灭。实物验证照片如图 10、图 11 和图 12 所示。

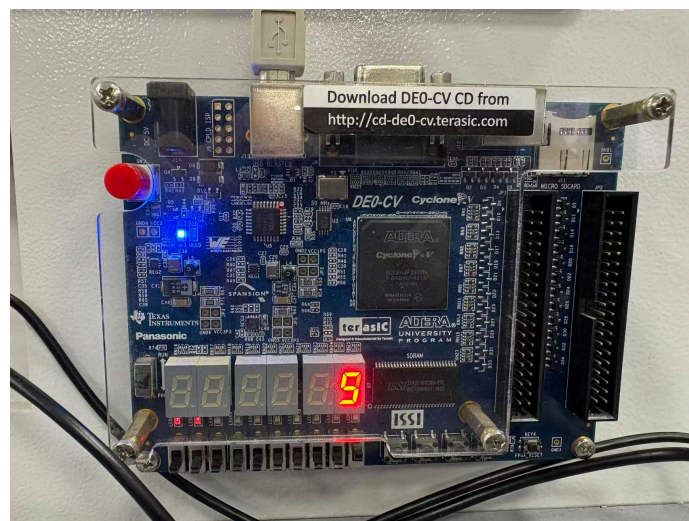


图 10 学号序列模式硬件验证实物图——显示字符“9”

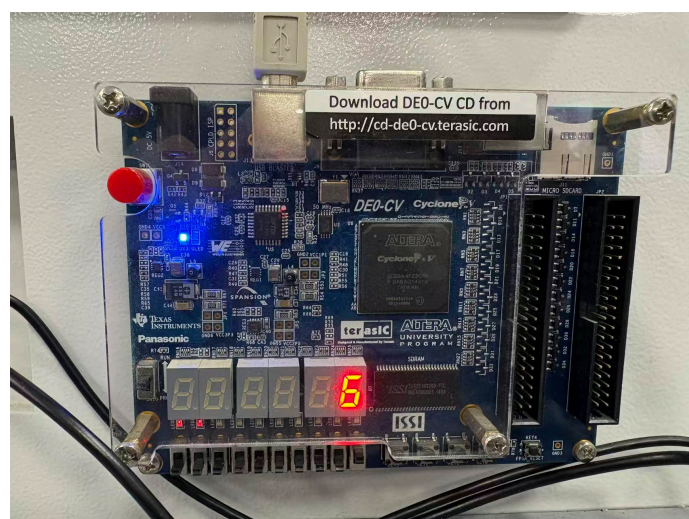


图 11 学号序列模式硬件验证实物图——显示字符“6”

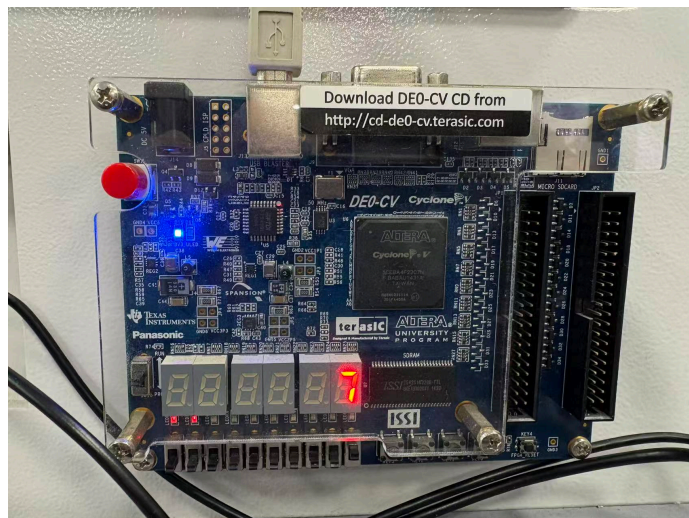


图 12 学号序列模式硬件验证实物图——显示字符“7”

两种模式均可通过拨动 SW0 开关实时切换，切换响应即时，运行稳定，实验结果与仿真完全吻合，硬件验证通过。

六、实验过程中的问题

1. 正弦波 ROM 初始化文件（.mif 或 .hex 格式）的数据格式与 Quartus II 要求不匹配，导致综合时报错无法识别 ROM 内容。检查后发现是十六进制前缀格式问题，修正为标准的 @address 格式后问题解决。
2. 学号 ROM 的数据宽度初始设置为 8 位，但实际只需 4 位即可表示 0～9、A、B、F 等字符编码。虽然不影响功能但浪费了 ROM 资源，重新配置 ROM 参数将数据宽度改为 4 位后优化了资源占用。
3. 仿真时由于分频系数过大（50 000 000），计数器需要极长时间才能完成一次完整的 ROM 扫描周期，导致仿真效率低下。通过在仿真测试平台中将分频系数临时缩小为 4，大幅缩短了仿真时间，同时保证了逻辑功能的完整性验证。下载前再将参数恢复为 50 000 000 以保证正确的 1 Hz 输出频率。
4. 模式切换时发现数码管存在短暂残影（ghosting）现象，即在切换瞬间上一个模式的输出残留约一个时钟周期。分析原因是模式切换信号与时钟边沿对齐不佳所致，通过在切换路径上加入同步寄存器，消除了残影现象。

七、心得体会

1. 本次实验深入学习了 ROM 在 FPGA 设计中的应用，认识到 ROM 本质上是一种查表机制，通过预先存储的数据配合地址扫描即可实现任意复杂的波形生成和序列显示功能。这种“存储 + 寻址”的设计思想在 DDS（直接数字频率合成）等领域有广泛应用。
2. 参数化分频器的设计使得同一个模块可以通过改变 generic 参数适应不同的分频需求，极大提高了代码的可复用性和灵活性。这种良好的编程习惯值得在后续设计中坚持。
3. 通过多模式切换系统的设计与调试，加深了对组合逻辑与混合逻辑设计的理解，也体会到硬件设计中信号同步和毛刺消除的重要性。同时进一步熟悉了 Quartus II 中 ROM IP 核的配置方法和.mif 初始化文件的编写规范，为后续更复杂的数据处理类系统设计积累了经验。